Problem Set 3

Name: Nicolas Iskandar

Collaborators: None

Problem 3-1.

(a) Hashes:

k	47	61	36	52	56	33	92
$(10k+4) \bmod 7$	5	5	0	6	4	5	0

Hash table:

0	$36 \to 92$
1	$\varnothing$
2	$\varnothing$
3	$\varnothing$
4	56
5	$47 \to 61 \to 33$
6	52

(b) Checking c from 7 upward because there are 7 keys and only 7 slots. Collisions occur for all $c < 13$; for $c = 13$, the hashes are distinct. Hence, the smallest $c = 13$. The Python code I used:

```python
A = [47, 61, 36, 52, 56, 33, 92]
slot = set()

for c in range(7, 100):
    for item in A:
        val = ((10 * item + 4) % c) % 7
        if val in slot:
            slot = set()
            break
        slot.add(val)

    if slot:
        print(c)
        break
```

Problem 3-2.

(a) The family $h_{ab}(k) = (ak + b) \bmod n$ is linear in k modulo n. Notice that $(ak + b) \bmod n$ only depends on $k \bmod n$, not on the exact value of k. So if two keys are congruent mod n, they collide under *every* function in the family, regardless of which a, b are chosen.

Choose $k_1 \equiv k_2 \pmod{n}$ (e.g., 1 and $n + 1$). Then for any a, b, $(ak_1 + b) \bmod n = (ak_2 + b) \bmod n$, so they are guaranteed to collide.

(b) The family $h_a(k) = \lfloor kn/u \rfloor + a \bmod n$ first buckets k into one of n contiguous ranges of the ID space (via $\lfloor kn/u \rfloor$), then shifts by a. Since $u \gg n$, many *consecutive* IDs fall into the very same range no matter what a is, because the shift and the mod-n reduction don't change which bucket-index results once two keys already share the same $\lfloor kn/u \rfloor$ value.

Choose $k_1 = 0, k_2 = 1$ (or $1, 2$ if IDs must be positive). Since $u \geq 2n$, $\lfloor kn/u \rfloor = 0$ for both, so $h_a(0) = h_a(1)$ for all a. Guaranteed.

(c) This family's defining property is that, for *any* two distinct keys, the probability of a collision over a uniformly random choice of a, b is at most $1/n$, where n is the number of buckets. Because this bound applies uniformly to every possible pair of keys, there is no clever choice of k_1, k_2 that Rony and Tiri can make in advance to beat it, unlike parts (a) and (b), where the structure of the family let an adversary force a guaranteed collision.

The family is universal, so for any two distinct keys the collision probability is at most $1/n$. They cannot guarantee a collision; the highest possible probability is $1/n$.

Problem 3-3.

(a) Identifier length $= 16\lceil \log_4(\sqrt{n}) \rceil = O(\log n)$ ASCII characters, which is $O(\log n)$ bits, small enough to fit in a constant number of machine words. Treating each identifier as an integer, the range of possible values is polynomially bounded in n, so radix sort applies directly.

Interpret as an integer in $[0, n^{O(1)})$ and radix-sort in $\Theta(n)$.

(b) The ages are bounded by a fixed constant (8×10^5) that does not grow with n. Whenever the range of possible key values is constant, counting sort gives linear time directly, without needing radix sort's extra machinery.

Ages are integers in the constant range $[0, 8 \cdot 10^5]$; use counting sort in $\Theta(n)$.

(c) The thicknesses are rationals of the form m/n^3. Multiplying every value by n^3 converts them into integers in the polynomially-bounded range $[0, 4n^3]$, which is again exactly the setting where radix sort achieves $\Theta(n)$ time.

Thickness $\times n^3$ gives integer $m \in [0, 4n^3]$; radix sort in $\Theta(n)$.

(d) Here there is no numeric key at all, only a pairwise comparison oracle. This puts us in the comparison model of sorting, whose worst-case lower bound is $\Theta(n \log n)$.

Merge sort achieves this bound and is therefore optimal; no algorithm relying solely on comparisons can do asymptotically better.

Only pairwise comparisons are available; use merge sort in $\Theta(n \log n)$, which is optimal in the comparison model.

Problem 3-4.

(a) The key idea is that for each box b_i, its only possible partner is the unique value $r - b_i$ (since all ream counts are distinct). A hash table lets us look up whether that partner exists, and if so, retrieve its index in expected $O(1)$ time, after which checking the closeness condition is $O(1)$.

Build a hash table mapping $b_i \mapsto i$ in expected $O(n)$. For each i, look up $r - b_i$; if found with j and $|i - j| < n/10$, return True. Expected $O(n)$.

(b) Since hashing only gives an *expected*-time guarantee, achieving worst-case $O(n)$ requires a deterministic approach based on sorting.

Prune. Discard any box with $b_i \geq r$: since every b_i is a positive integer, such a box would require its partner to satisfy $b_j \leq 0$ to sum to r, which is impossible. This filtering pass takes $O(n)$ time and cannot remove any valid pair.

Bound and sort. Every surviving value satisfies $b_i < r < n^2$, a polynomial range in n. Radix-sort the tuples (b_i, i) by b_i (retaining original indices, needed later for the closeness check) into an array A; since the keys are bounded by $O(n^2)$, this runs in worst-case $\Theta(n + n \log_n(n^2)) = \Theta(n)$ time, unlike hashing.

Two-pointer sweep. Initialize $i = 0$, $j = |A| - 1$. Repeat until $i = j$:

- If $A[i].b + A[j].b = r$: a valid sum has been found. Check $|A[i].x - A[j].x| < n/10$; if satisfied, return `True`.
- If $A[i].b + A[j].b < r$: even paired with the largest remaining value, $A[i]$ cannot reach r, so no smaller partner can either; increment i.
- If $A[i].b + A[j].b > r$: symmetrically, $A[j]$ cannot be part of a valid pair with anything remaining; decrement j.

Each iteration does $O(1)$ work and permanently eliminates one array position, so the sweep runs in $O(n)$.

Correctness. The pruning step never discards a box that could be part of a valid pair. In the sweep, each discarded pointer position is provably incapable of forming a valid sum with *any* remaining element, so no valid pair is ever skipped; if the pointers meet without a close matching sum being found, no such pair exists.

Running time. Pruning, sorting, and sweeping each take $O(n)$ worst-case time, for a total of $O(n)$, with no reliance on randomization.

Problem 3-5.

(a) We maintain a 26-entry frequency vector for each length-k substring of A. Rather than recomputing each window's frequency vector from scratch (which would cost $O(k)$ per window and $O(|A|k)$ overall), we slide the window across A: at each step we decrement the count of the character leaving the window and increment the count of the character entering it, an $O(1)$ update. Each resulting frequency vector is stored as a key in a hash table, mapped to how many windows produced that exact vector. To answer a query for string B, we compute B's own frequency vector in $O(k)$ time and look it up in the table, the stored count is exactly the anagram substring count, or zero if the vector never appeared.

For each length-k substring of A, maintain a 26-entry frequency vector. Build all vectors via a sliding window in $O(|A|)$ and store each vector's multiplicity in a hash table. Query: compute B's vector in $O(k)$ and look it up.

(b) This is simply a batched version of part (a): build the data structure once on T, then reuse it for each of the n queries.

Build the data structure from (a) on T in $O(|T|)$; for each S_i, compute its vector in $O(k)$ and look up. Total expected time $O(|T| + nk)$.